

ALMA MATER STUDIORUM · UNIVERSITÀ DI BOLOGNA

TODO Dipartimento di Informatica — Scienza e Ingegneria
Corso di Laurea Triennale in Ingegneria e Scienze informatiche

TODO

TODO

Relatore:
Chiar.ma Prof.
TODO

Presentata da:
TODO

Correlatore:
Chiar.mo Prof.
TODO

Sessione MESE ANNO
Anno Accademico 20TODO/20xx

Indice

1	Introduzione	11
1.1	Iot (Internet of Things)	11
1.2	Publish/Subscribe	11
1.3	La difficoltà del percorso ottimo	12
1.3.1	Grafo	12
1.3.2	Centralizzazione	13
1.4	Problema di ottimizzazione	14
1.4.1	Rilassamento Continuo	14
1.4.2	Rilassamento Lagrangiano	15
1.5	Ambiente di sviluppo	19
2	Descrizione del Problema	21
2.1	Scenario del problema	21
2.2	Simboli Globali	22
2.3	Prima formulazione del problema	23
2.4	Seconda formulazione	24
2.5	Rilassamento lagrangiano della seconda formulazione	26
2.5.1	Aggiornamento Lambda	28
3	Implementazione	29
3.1	Upper bound	29
3.2	Knapsack	29
3.3	Scelta step	29
3.4	Possibile algoritmo di mostraggio risultato o distribuito o correzione di quello del tipo	29

Elenco delle figure

1.1	Immagine di funzionamento di una rete Publish/Subscribe	12
1.2	Immagine di un grafo con costi e capacità degli archi	13
1.3	Immagine di una funzione lagrangiana con inviluppo	17
1.4	Valore di $L(\bar{\lambda})$ corrispondente a una $\bar{\lambda}$ scelta	17
1.5	Immagine dell'ambiente di sviluppo di CPLEX Optimization Studio	19
1.6	Immagine dell'ambiente di sviluppo di Google Colab	19
2.1	Immagine di una rete con dei Publisher (sensori), dei Broker e dei Subscriber	21

Elenco delle tabelle

Sommario

TODO IoT (Internet of Things) è l'acronimo utilizzato per indicare la sfera tecnologica che ha alla base l'utilizzo di oggetti intelligenti. Per oggetti intelligenti si intendono quei dispositivi, macchine o attrezzature che si possono connettere a internet per trasmettere, ricevere dati o elaborarli. All'interno di questa sfera, uno dei metodi di comunicazione più utilizzati è quello basato su un modello Publish/Subscribe che consente lo scambio di dati tra dispositivi della rete senza la necessità di connessioni dirette tra loro. Per funzionare si utilizza il protocollo MQTT, uno dei protocolli più diffusi per lo scambio di messaggi, il quale si basa su una struttura con 3 tipologie di dispositivi connessi: Publisher, Subscriber e Broker. I Publisher una volta ottenuti dei dati tramite sensori di vario genere, li pubblica, rendendoli disponibili nella rete. I dati per spostarsi all'interno della rete utilizzano i broker cioè vari punti/server di connessione, i quali permettono il passaggio di dati per poi infine arrivare ai Subscribers per la lettura e l'elaborazione. Questa tesi riprende il lavoro effettuato da un collega nell'anno 21-22 approfondendo e sviluppando i casi di test con particolare attenzione a quelle che sono le caratteristiche che possono portare criticità al calcolo della soluzione ottima di percorso per il flusso dei dati. La tesi del collega si basa sull'idea di costruire un sistema distribuito sfruttando le proprietà matematiche del rilassamento Lagrangiano, dove ogni nodo conosce solamente i propri vicini.

1 Introduzione

1.1 Iot (Internet of Things)

Il termine Iot nasce come acronimo per indicare quella sfera tecnologica moderna che si basa sull'utilizzo di dispositivi "intelligenti". Quest'ultima ha avuto particolare rilevanza nell'ultimo periodo storico, andando a rivoluzionare il modo in cui si raccolgono e si processano le informazioni sia in ambito domestico sia industriale.

Un dispositivo definito intelligente o più comunemente chiamato "smart" è un dispositivo in grado di connettersi alla rete per poter svolgere diverse funzioni in cui tra le più importanti è presente lo scambio di informazioni.

In questa tesi ci si sofferma particolarmente sui "sensori", dispositivi chiave per lo scambio di messaggi in una rete IoT. Un sensore è un dispositivo che rileva un cambiamento nel mondo fisico, lo traduce in un segnale digitale e poi lo diffonde ad altri dispositivi, i quali lo utilizzeranno per prendere decisioni.

Una possibile criticità risiede proprio in questa parte del funzionamento, se una decisione deve essere presa con tempestività, anche la rete di trasmissione dovrà essere veloce e efficiente.

1.2 Publish/Subscribe

Tra le metodologie più famose di trasmissione di messaggi, quella che si intende approfondire è il Publish/Subscribe, un paradigma particolarmente adatto all'IoT, nato come sostituzione del classico Client-Server. In questo schema i dispositivi si dividono in Publisher e Subscriber collegati tra loro tramite dei nodi/server di trasmissione definiti Broker.

In questo modello ognuno ha il suo ruolo: i Publisher devono diffondere nella rete le informazioni in loro possesso, i Broker devono ritrasmettere queste informazioni senza permettere danneggiamenti o perdita delle stesse e i Subscriber sono i ricevitori o i richiedenti. Per quanto riguarda il meccanismo alla base è molto semplice: il Publisher invia messaggi etichettati con un topic, il Broker li riceve e li inoltra solamente ai Subscriber iscritti a quel topic.

Questo approccio permette un disaccoppiamento tra nodo di partenza e di arrivo, in quanto le due parti non si conoscono direttamente e non devono per forza essere attive contemporaneamente per garantire il successo della comunicazione.

Il protocollo standard per l'implementazione di questo modello è l'MQTT, progettato per essere leggero e robusto anche su reti instabili ma che presenta alla base una criticità importante: essendo il Broker il nodo centrale di comunicazione, la sua irraggiungibilità diventa rischio di interruzione per l'intera rete. Per ovviare al problema, si è optato per la creazione di infrastrutture con più Broker configurati per lavorare e apparire come uno solo, in modo che nel caso di fallimento di un server, i rimanenti possano continuare il lavoro.

Facendo riferimento al paragrafo precedente si può associare il ruolo di Publisher

ai sensori e il ruolo di Subscriber a tutti quei dispositivi che necessitano del valore rilevato.

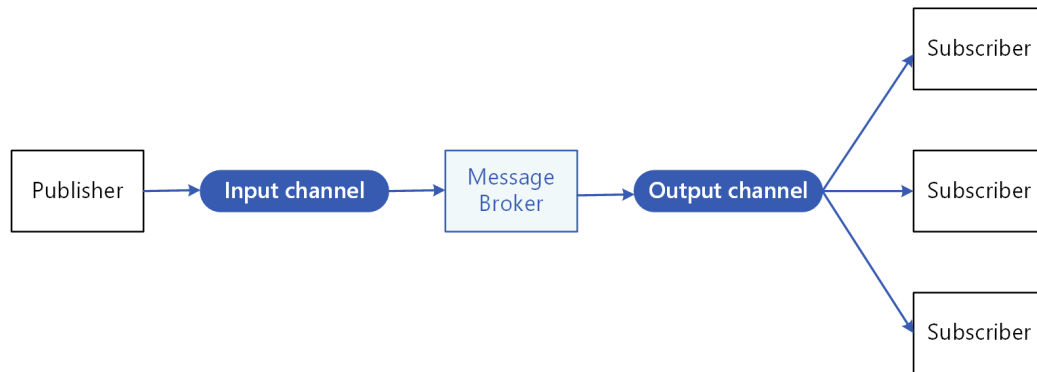


Figura 1.1: Immagine di funzionamento di una rete Publish/Subscribe

1.3 La difficoltà del percorso ottimo

In un sistema moderno, trovare il percorso ottimo per la trasmissione dei dati è cruciale, infatti una scelta non ottimizzata potrebbe ritardare o addirittura impedire la ricezione delle informazioni, compromettendo il funzionamento dei dispositivi che dipendono da esse. Per questo motivo è necessario individuare il metodo più efficiente per rappresentare la rete e per calcolare il percorso ottimale.

1.3.1 Grafo

Uno dei metodi migliori per poter descrivere una rete di dispositivi connessi è tramite un grafo. Usando questa struttura, i dispositivi vengono rappresentati tramite dei nodi e i collegamenti tramite degli archi.

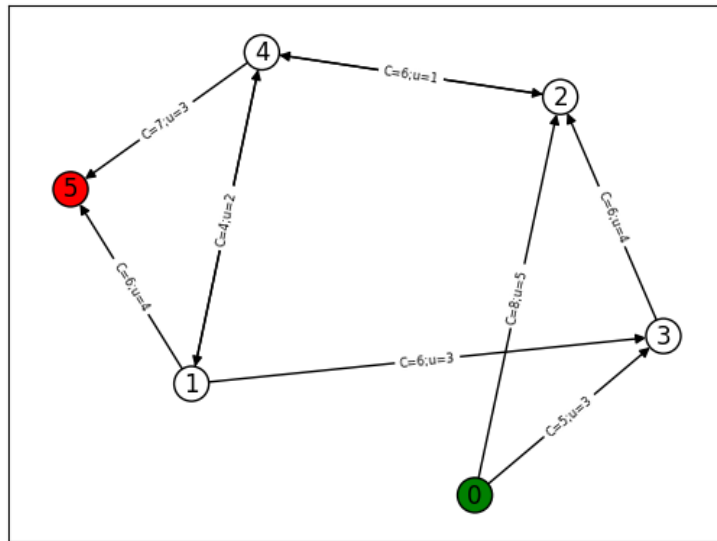


Figura 1.2: Immagine di un grafo con costi e capacità degli archi

L'uso di un grafo risulta una scelta efficiente perché, per reti semplici, consente di mostrare in modo naturale sia i componenti della rete sia le relazioni tra essi e grazie alla sua flessibilità permette inoltre di aggiungere varie proprietà agli elementi rappresentati, rendendo possibile modellare scenari complessi con facilità. **TODO Capire se mi conviene parlare della centralizzazione e del 2p2 dato che lavoro più in un ambito non decentralizzato nel mio caso**

1.3.2 Centralizzazione

Un altro aspetto fondamentale del problema riguarda la centralizzazione dell'elaborazione della rete. In un caso reale, calcolare qual è il percorso ottimo necessita di una notevole potenza di calcolo e, soprattutto, bisogna che ci sia la garanzia di possedere informazioni complete sulla composizione della rete. Da qui nasce l'idea di non affidarsi più esclusivamente ad un server centrale ma di distribuire il calcolo su tutti i dispositivi della rete rendendola autonoma sia nella ricerca del percorso ottimo sia nella gestione dei guasti in caso di malfunzionamento di un canale.

Peer-To-Peer

Ciò è reso possibile sfruttando un protocollo Peer-to-Peer, ovvero un protocollo che tratta i dispositivi in comunicazione sia come Client sia come Server, permettendogli dunque di scambiarsi richieste di risorse o servizi senza dover passare da

terzi. **TODO origini del p2p, strutturati e non strutturati, problemi legati al p2p.**

1.4 Problema di ottimizzazione

Il problema descritto è così definito come un problema di ottimizzazione, problema che richiede di trovare la migliore soluzione possibile fra tutte le soluzioni ammissibili, massimizzando o minimizzando una funzione definita obiettivo e rispettando i vincoli imposti.

Questo tipo di problema è descrivibile utilizzando un modello matematico:

$$\min/\max \sum_i c_i x_i \quad (1.1)$$

vincoli:

$$Ax \geq b \quad (1.2)$$

$$Bx \geq d \quad (1.3)$$

$$x \in \{0, 1\} \quad (1.4)$$

Con c il vettore dei costi, x la variabile decisionale, A e B le matrici dei coefficienti del problema, b e d i vettori delle costanti per i vincoli.

Molto spesso quando si tratta di problemi di questo tipo, si possono riscontrare delle difficoltà nella risoluzione, per questo viene adottata la tecnica del rilassamento dei vincoli che consente di semplificare il calcolo ottenendo un limite inferiore per la soluzione del problema originale.

1.4.1 Rilassamento Continuo

Il rilassamento più intuitivo è quello continuo, dove se il problema presenta la variabile decisionale come intera, è possibile costruire un problema di ottimizzazione equivalente, abbandonando il vincolo di interezza della variabile e permettendole di assumere un qualsiasi valore reale all'interno di un intervallo continuo derivato dal vincolo originale.

Questo approccio trasforma il problema intero in un problema di programmazione lineare continua, per il quale esistono algoritmi efficienti. Lo spazio delle soluzioni ammissibili, tuttavia, è più ampio e il valore ottimo del problema rilassato potrebbe non essere sempre applicabile al problema originale.

$$x \in \{0, 1\} \rightarrow 0 \leq x \leq 1 \quad (1.5)$$

1.4.2 Rilassamento Lagrangiano

Un rilassamento particolarmente utile è quello lagrangiano, che permette di rilassare i vincoli computazionalmente dispendiosi aggiungendoli alla funzione obiettivo e associando a ognuno di essi un moltiplicatore di Lagrange. Quest'ultimo funge da penalità: più ci si allontana dal rispetto del vincolo, maggiore sarà il peggioramento del valore della funzione obiettivo, allontanandolo dall'ottimo. Il risultato è una nuova funzione obiettivo, detta lagrangiana, che dipende sia dalle variabili originali del problema sia dai moltiplicatori di Lagrange, e la cui risoluzione fornirà i valori ottimi di entrambi.

Spiegazione matematica

Si considera un problema di minimizzazione P:

$$z(P) = \min cx \quad (1.6)$$

vincoli:

$$Ax \geq b \quad (1.7)$$

$$x \in \{0, 1\} \quad (1.8)$$

Con c il vettore dei costi, x la variabile decisionale, A la matrice dei coefficienti del problema, b il vettore delle costanti per il vincolo.

Con il rilassamento lagrangiano si rilassa il vincolo $Ax \geq b$ aggiungendo una penalità λ non negativa e inserendolo all'interno della funzione obiettivo.

$$z(P) = \min cx, Ax - b \geq 0 \quad (1.9)$$

$\downarrow$

$$L(\lambda) = \min cx - \lambda(Ax - b) \quad (1.10)$$

vincoli:

$$x \in \{0, 1\} \quad (1.11)$$

Il motivo del segno $-$ è dovuto al fatto che il vincolo è soddisfatto quando assume un valore maggiore o uguale a 0, quindi dato che λ è definita non negativa, se il vincolo non viene soddisfatto come nel caso in cui sia negativo, sottrarlo dalla funzione obiettivo ne aumenta il valore, allontanandolo dal minimo ricercato.

Il valore ottimale del problema originale $z(P)$ sarà maggiore o al massimo uguale al valore ottimale del problema rilassato $L(\lambda)$.

$$z(P) \geq L(\lambda), \quad \forall \lambda \geq 0 \quad (1.12)$$

Questo perché non avendo l'obbligo di rispettare il vincolo, che è stato rilassato, lo spazio delle soluzioni ammissibili è più ampio e quindi si possono trovare soluzioni che il problema originale non può considerare corrette, le quali, avendo meno costrizioni, avranno un valore più basso o al massimo uguale alla soluzione dell'originale.

1. Si prende una x^* che è soluzione ottima di $z(P)$ quindi è ammissibile e il vincolo è rispettato.
2. Per $\lambda \geq 0$ e $Ax^* - b \geq 0$ si ha:

$$cx^* \geq cx^* - \lambda(ax^* - b) \quad (1.13)$$

3. x^* è anche soluzione ammissibile per il problema rilassato quindi esiste sicuramente una soluzione di $L(\lambda)$ che sarà o minore o uguale al valore di x^* :

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq cx^* - \lambda(Ax^* - b) \quad (1.14)$$

4. Unendo le disuguaglianze:

$$z(P) = cx^* \geq cx^* - \lambda(ax^* - b) \geq L(\lambda) \rightarrow z(P) \geq L(\lambda) \quad (1.15)$$

Grazie a questa proprietà, si può pensare di massimizzare la funzione lagrangiana per ottenere il migliore limite inferiore possibile per il problema originale e si può descrivere con la formula del duale lagrangiano:

$$z(D_L) = \max_{\lambda \geq 0} [L(\lambda)] \quad (1.16)$$

Una delle tecniche che può essere usata per la ricerca della soluzione del duale lagrangiano è il metodo del subgradiente.

Metodo del subgradiente

Il metodo del subgradiente è un metodo iterativo usato per massimizzare una funzione concava, come quella lagrangiana, con lo scopo di ottenere il migliore limite inferiore per il problema originale. Nello specifico, si può utilizzare per calcolare i moltiplicatori di Lagrange associati ai vincoli rilassati che forniscono il risultato $L(\lambda)$ più alto.

La funzione lagrangiana non è perfettamente una curva liscia ma è composta di segmenti, ciascuno corrispondente a una soluzione ottima $\bar{x}$ per il λ specificato.

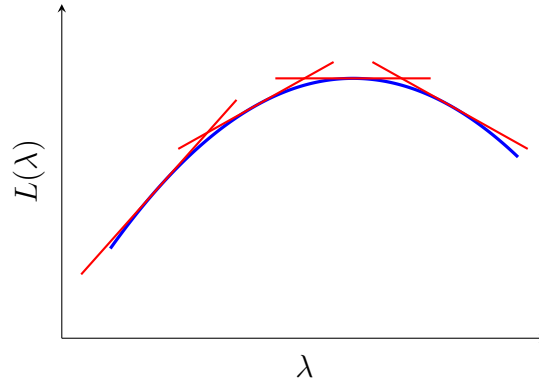


Figura 1.3: Immagine di una funzione lagrangiana con involucro

Data una λ , si ottiene una $\bar{x}$ tale che:

$$L(\bar{\lambda}) = \min(cx - \bar{\lambda}(Ax - b)) = c\bar{x} - \bar{\lambda}(A\bar{x} - b) \quad (1.17)$$

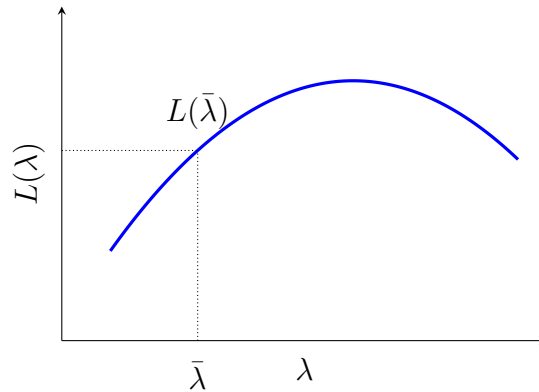


Figura 1.4: Valore di $L(\bar{\lambda})$ corrispondente a una $\bar{\lambda}$ scelta

Riprendendo la soluzione $\bar{x}$ derivata dalla $\bar{\lambda}$ data, si può ridefinire $L(\lambda)$, cioè il valore ottimo in una qualsiasi altra λ generica, come o minore o uguale della soluzione trovata ora:

$$L(\lambda) = \min(cx - \lambda(Ax - b)) \leq c\bar{x} - \lambda(A\bar{x} - b) \quad (1.18)$$

Questo perché la disequazione confronta elementi diversi, nel lato sinistro è presente una x ottima per la nuova λ , mentre nel lato destro è presente la $\bar{x}$, ottima per $\bar{\lambda}$ ma non necessariamente per λ . Poiché le si stanno valutando con la stessa λ , allora per definizione, la x ottima darà un valore minore o, nel caso in cui anche $\bar{x}$

sia ottima per λ , uguale.

Sottraendo ora l'equazione (1.17) dalla (1.18) si ottiene:

$$L(\lambda) - L(\bar{\lambda}) \leq c\bar{x} - \lambda(A\bar{x} - b) - (c\bar{x} - \bar{\lambda}(A\bar{x} - b)) \quad (1.19)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq -\lambda(A\bar{x} - b) + \bar{\lambda}(A\bar{x} - b) \quad (1.20)$$

$\downarrow$

$$L(\lambda) - L(\bar{\lambda}) \leq (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.21)$$

$\downarrow$

$$L(\lambda) \leq L(\bar{\lambda}) + (A\bar{x} - b)(-\lambda + \bar{\lambda}) \quad (1.22)$$

Si definisce $s = -(A\bar{x} - b)$ e si trasforma l'equazione:

$$L(\lambda) \leq L(\bar{\lambda}) + s(\lambda - \bar{\lambda}) \quad (1.23)$$

L'obiettivo della risoluzione del duale lagrangiano è, a ogni iterazione, di far crescere $L(\lambda)$ fino a trovare il valore massimo. In termini matematici si vuole che $L(\lambda)$ sia maggiore di $L(\bar{\lambda})$ e che quindi la loro differenza sia positiva. L'unico modo di ottenere questo risultato è di avere il termine: $s(\lambda - \bar{\lambda}) > 0$ e che quindi sommandolo a $L(\bar{\lambda})$ ne aumenti il valore. Per garantire che ciò accada si opta per spostare λ rispetto a $\bar{\lambda}$ usando lo stesso verso del subgradiente s in modo che:

$$\lambda = \bar{\lambda} + \theta s \quad (1.24)$$

$\downarrow$

$$\lambda - \bar{\lambda} = \theta s \quad (1.25)$$

$$s(\lambda - \bar{\lambda}) = s(\theta s) = \theta s^2 \quad (1.26)$$

Di conseguenza per un $s \neq 0$ e per un passo $\theta > 0$ il valore risultante sarà sicuramente positivo e porterà ad una λ maggiore rispetto a $\bar{\lambda}$.

Anche la scelta del parametro θ è fondamentale, un passo troppo grande potrebbe far saltare la soluzione ottima portando alla divergenza, mentre un passo troppo corto potrebbe convergere alla soluzione ottima troppo lentamente.

1.5 Ambiente di sviluppo

Esistono molti ambienti di sviluppo che permettono la risoluzione di problemi di ottimizzazione, ma quelli su cui ci si vuole soffermare in questa tesi sono CPLEX Optimization Studio e Google Colab.

CPLEX Optimization Studio è un software sviluppato da IBM con l'obiettivo di permettere la risoluzione di problemi di ottimizzazione in maniera intuitiva senza l'obbligo di dover implementare i metodi risolutivi da parte dell'utente finale. Per garantire semplicità è richiesta solamente la struttura del problema, i dati e la configurazione di avvio. È un software potente ma presenta delle limitazioni: è particolarmente costoso in termini economici, usa un linguaggio di programmazione molto specifico, non facilmente riconoscibile a prima vista da un utente inesperto e modellare problemi di rilassamento lagrangiano al suo interno è complesso.

Google Colab invece è un ambiente Python online scelto per diversi vantaggi: le librerie sono già installate o facilmente installabili, i file sono salvati nel cloud, quindi accessibili da più dispositivi e la modalità a celle permette di eseguire porzioni di codice senza dover rieseguire tutto ogni volta. La scelta di Python è dovuta sia a delle difficoltà tecniche nel far collaborare l'ambiente di sviluppo CPLEX con Java, sia al fatto che si parla di un ambiente estremamente flessibile e in quanto tale, permette di importare numerose librerie per la semplificazione dei calcoli. È grazie a questa flessibilità che l'ambiente di Google Colab permette di modellare anche problemi di ottimizzazione complessi, come il rilassamento lagrangiano, e permette la visualizzazione delle casistiche tramite librerie grafiche come Networkx e Matplotlib.pyplot. Lo svantaggio principale di questo ambiente è la necessità di implementare i metodi risolutivi che, in molti casi, non presentano aiuti da librerie esterne o implementazioni pronte e quindi vanno scritti da zero sulla base della teoria.

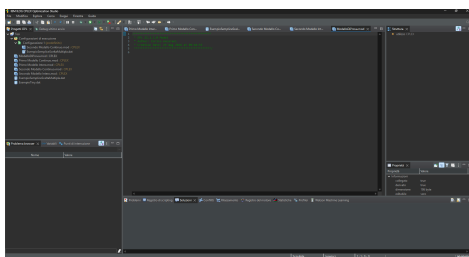


Figura 1.5: Immagine dell'ambiente di sviluppo di CPLEX Optimization Studio

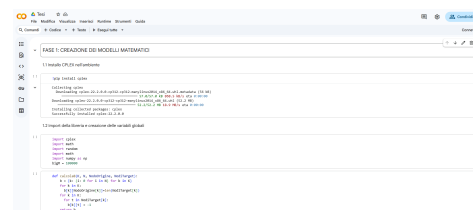


Figura 1.6: Immagine dell'ambiente di sviluppo di Google Colab

2 Descrizione del Problema

2.1 Scenario del problema

TODO, ricontrollare e cambiare in base a quale sarà la scelta per l'obiettivo finale della tesi Il problema che si vuole approfondire nello specifico è quello di una rete IoT in cui si vogliono scambiare informazioni tra generatori di dati e ricevitori. La rete è caratterizzata da una struttura Publish/Subscribe che, come definita in precedenza, presenta al suo interno dei nodi di comunicazione definiti Broker, sensori che svolgono il ruolo di generatori dei dati (Publisher) e utenti finali che svolgono il ruolo di ricevitori (Subscriber).

L'obiettivo della tesi è sviluppare, prendendo come riferimento il lavoro di un collega, un algoritmo efficiente per il calcolo del costo del percorso ottimale. **TODO da espandere in caso di altre cose da fare.**

Per modellare correttamente il problema è necessario considerare un caso reale, quindi con diversi fattori in gioco:

- Capacità del canale (quante informazioni possono passare insieme nello stesso collegamento nello stesso momento).
- Costo del canale (quanto è dispendioso in termini economici o computazionali utilizzare quel determinato percorso).
- Peso di un'informazione (quanta banda occupa la singola informazione all'interno del canale).

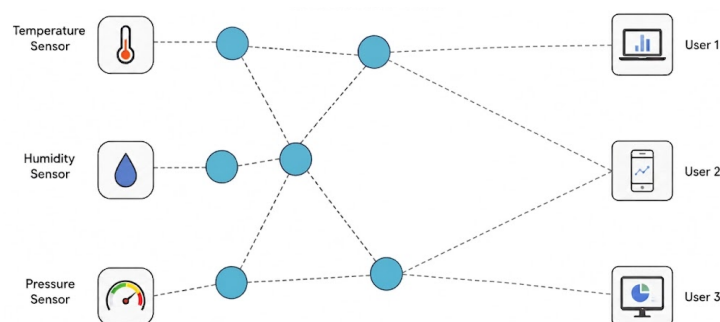


Figura 2.1: Immagine di una rete con dei Publisher (sensori), dei Broker e dei Subscriber (utenti finali)

2.2 Simboli Globali

Per poter iniziare una transizione del problema dal mondo reale a una formulazione matematica è necessario mappare le variabili reali in un contesto informativo/matematico:

- K , è l'insieme contenente le informazioni da inviare o ricevere.
- N , è l'insieme contenente tutti i nodi della rete.
- *Publisher*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe fanno parte dei generatori delle informazioni.
- *Broker*, è l'insieme contenente i nodi di mezzo che non devono né generare né consumare le informazioni ma solo ritrasmettere.
- *Subscriber*, è l'insieme contenente i nodi che secondo il modello Publish/Subscribe devono ricevere le informazioni.
- A , è l'insieme contenente gli archi della rete quindi i canali di comunicazione.
- $NodoOrigine_k$, è l'insieme contenente per ogni informazione k il suo nodo di origine.
- $NodiTarget_k$, è l'insieme contenente per ogni informazione k i nodi che richiedono quella informazione.
- b_i^k , è una variabile che a seconda del nodo i e della informazione k mi indica il ruolo di i rispetto a k : se è ≥ 0 allora mi indica che è un generatore e per quanti *NodiTarget* sta producendo k , se è 0 mi indica che è un nodo di trasmissione e invece se è -1 mi indica che è un nodo richiedente di k .
- a_k , è una variabile che mi indica per ogni informazione k quanta banda occupa all'interno di un canale.
- c_{ij} , è l'insieme contenente per ogni arco i, j il suo costo.
- u_{ij} , è l'insieme contenente per ogni arco i, j la sua capacità.
- Lt_i , è l'insieme contenente per ogni nodo i tutti i nodi raggiungibili da i .
- Lr_i , è l'insieme contenente per ogni nodo i tutti i nodi da cui si può raggiungere i .

- x_{ij}^k , è una variabile decisionale che mi indica quanti subscriber richiedenti l'informazione k sto servendo usando l'arco i, j .
- e_{ij}^k , è una variabile decisionale che mi indica se per il trasporto di una informazione k attivo l'arco i, j .
- M , è una variabile di valore molto alto usata per definire dei vincoli.

2.3 Prima formulazione del problema

TODO capire se descrivere le problematiche di questa formulazione come il crollo in caso di nodo saturo Una prima formulazione può essere descritta con uno schema minimum-cost maximum-flow, un problema comunemente noto nell'ambito dell'ottimizzazione dove viene richiesto di soddisfare il maggior numero di richieste possibili minimizzando i costi nel farlo.

Funzione Obiettivo:

$$\text{minimizza } \sum_{k \in K} \sum_{i \in N} \sum_{j \in Lr_i} c_{ji} x_{ji}^k \quad (2.1)$$

(2.1): Per ogni informazione k , per ogni nodo i , si somma il costo degli archi entranti in i moltiplicati per il numero di Subscriber serviti su quell'arco. In pratica il costo totale dipende da quante volte l'informazione k sta passando per l'arco i, j nello stesso momento.

Vincoli:

$$\sum_{j \in Lr_i} x_{jt}^k = 1 \quad \forall t \in NodiTarget_k, k \in K \quad (2.2)$$

(2.2): Sommando le informazioni in entrata al Subscriber il risultato deve corrispondere a 1. Questo vincolo permette di garantire che ogni subscriber riceva l'informazione che ha richiesto, né di meno né di più. Questo deve valere per ogni subscriber e per ogni informazione richiesta.

$$\sum_{j \in Lr_i} x_{ji}^k = \sum_{j \in Lt_i} x_{ij}^k \quad \forall i \in Broker, k \in K \quad (2.3)$$

(2.3): Sommando il numero di Subscriber serviti con gli archi entranti nel Broker i , il risultato deve essere uguale al numero di Subscriber serviti con gli archi uscenti. Questo vincolo garantisce che i è esclusivamente un nodo di passaggio e che non può generare o consumare informazioni.

$$M e_{ij}^k \geq x_{ij}^k \quad \forall (i, j) \in A, k \in K \quad (2.4)$$

Se sto soddisfacendo un subscriber attraverso l'arco allora questo per forza deve essere considerato attivo rispetto al transito del flusso k . Questo vincolo ci serve per legare la variabile e con la x in modo che se sto usando un arco allora è considerato per forza attivo. M è scelto come il numero totale dei nodi perché è impossibile in quanto nodi loro stessi che il numero di subscribers siano maggiori della somma dei nodi ($N \geq$ numero subscribers in quanto loro nodi stessi).

$$\sum_{k \in K} a^k (e_{ij}^k + e_{ji}^k) \leq u_{ij} \quad \forall i \in N, j \in Lt_i \quad (2.5)$$

Per ogni arco controllo che la sommatoria della banda occupata da tutti i flussi passanti da quell'arco in qualsiasi direzione non sia mai maggiore della capacità totale dell'arco. Questo vincolo mi permette di non superare mai il valore massimo della capacità di un arco controllando quando occupano tutti i flussi passanti da esso.

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.6)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.7)$$

La differenza sta nel primo è il caso intero, nel secondo è rilassamento continuo. In questa formulazione ci immaginiamo che ogni volta che una informazione viene inviata ne viene inviata una copia per ogni subscriber che deve soddisfare quindi usa più canale e costa di più.

2.4 Seconda formulazione

TODO capire se si vuole parlare del fatto che cerchiamo una formula distribuita o meno La seconda formulazione del problema viene definito in maniera diversa, infatti non ci preoccupiamo più di capire quante informazioni far passare da quali nodi ma decidiamo di gestire il problema decidendo se attivare o meno un arco. Funzione obiettivo:

$$\text{minimizza} \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k \quad (2.8)$$

Per ogni flusso k , sommiamo il costo dell'arco per il fatto se è attivo o meno per il trasporto di k , minimizzando cerchiamo di usare il numero minore di archi possibili e comunque soddisfare tutte le richieste. In questa formulazione non si ragiona come ogni invio di una informazione costa come un utilizzo intero ma l'informazione viene copiata e inviata in ogni nodo di mezzo portando a un costo che non

dipende da quante volte invio l'informazione attraverso un canale ma semplicemente se il canale è attivato per il trasporto di quella informazione. L'altro obiettivo della seconda formulazione è quello di avere una formula facilmente rilassabile con lagrange in modo da creare sotto problemi definibili locali, per problema locale intendiamo un problema che sia indipendente dagli altri e lavora solamente conoscendo la situazione dei suoi vicini e archi.

Vincoli:

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k = b_i^k \quad \forall i \in N - \{NodoOrigine_k\}, k \in K \quad (2.9)$$

Sommando i flussi in entrata e uscita dal nodo, il valore deve risultare con la funzione del nodo, ≥ 0 se genera flussi, 0 se è un broker quindi il numero di flussi non cambia e si mantiene, -1 se è un subscriber che consuma il flusso. La scelta di questo preciso vincolo viene fatta per accorpate il vincolo 18 e 19 che lasciati separati entrambi analizzavano più archi. (18 con la sommatoria) e per il rilassamento lagrangiano non può andare bene in quanto ha bisogno di rilassare un solo vincolo per creare poi tutti i vincoli indipendenti basanti su l'analisi di un solo arco alla volta. Questo vincolo ci serve per dichiarare che ognuno riceva la quantità di flusso richiesta e che i broker non possano eliminare o aggiungere flusso.

$$\sum_{k \in K} a^k e_{ij}^k \leq u_{ij} \quad \forall (i, j) \in A \quad (2.10)$$

Per ogni arco controllo che la somma di quanto occupano i flussi passanti da quell'arco non sia mai più grande della capacità massima dell'arco stesso. Questo vincolo ci serve per garantire che non si superi mai la capacità massima dell'arco.

$$e_{ij}^k \leq x_{ij}^k \leq e_{ij}^k |NodiTarget_k| \quad \forall (i, j) \in A, k \in K \quad (2.11)$$

Se l'arco è attivo allora deve per forza passare del flusso e nell'arco al massimo per ogni flusso k può passare un numero di flussi pari ai subscriber che deve soddisfare. Questo vincolo ci garantisce un legame tra l'attivazione di un arco e il flusso che passa al suo interno.

$$x_{ij}^k \geq 0 \quad \forall (i, j) \in A, k \in K \quad (2.12)$$

$$e_{ij}^k \in \{0, 1\} \text{ oppure } 0 \leq e_{ij}^k \leq 1 \quad \forall (i, j) \in A, k \in K \quad (2.13)$$

Dipende dal rilassamento intero o continuo.

2.5 Rilassamento lagrangiano della seconda formulazione

Andando ad applicare il rilassamento lagrangiano nella seconda formulazione si noterà come la formula cambierà molto e attraverserà vari passaggi nel farlo. Prima di tutto si definiscono le famose penalità classiche del rilassamento lagrangiano: λ_i^k con k informazione e i nodo. Questo succede perché si ha un vincolo 25 per ogni i e k quindi ci saranno lo stesso numero di penalità. Si crea il vincolo $= 0$

$$\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k = 0 \quad (2.14)$$

Da qui si crea la nuova funzione obiettivo lagrangiana:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \left(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k \right) \quad (2.15)$$

Il motivo della doppia sommatoria $\sum_{k \in K} \sum_{i \in N}$ è perché si vuole trovare il minimo considerando che il vincolo sia applicato e rispettato per tutte le k su tutti i nodi (quando abbiamo creato il vincolo era per via del $\forall i \in N, k \in K$) Il $+$ invece prima delle sommatorie è poiché il vincolo è un'uguaglianza, λ può assumere qualsiasi valore reale. In caso di violazione, il segno di λ si adatta al segno del vincolo non rispettato, garantendo che il termine di penalità $\lambda(\sum_{j \in Lt_i} x_{ij}^k - \sum_{j \in Lr_i} x_{ji}^k - b_i^k)$ sia sempre positivo e allontani la soluzione dal minimo.

Espandendo:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lr_i} x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k * b_i^k \quad (2.16)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \sum_{i, j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i, j \in A} \lambda_i^k x_{ij}^k - \sum_{k \in K} \sum_{j, i \in A} \lambda_i^k x_{ji}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k * b_i^k \quad (2.17)$$

La trasformazione delle sommatorie da $\sum_{k \in K} \sum_{i \in N} \lambda_i^k \sum_{j \in Lt_i} x_{ij}^k$ a $\sum_{k \in K} \sum_{i, j \in A} \lambda_i^k x_{ij}^k$ succede perché quando li moltiplico per la legge delle sommatorie diventa $\sum_{k \in K} \sum_{i \in N} \sum_{j \in Lt_i} \lambda_i^k x_{ij}^k$ ma pensandoci bene se per ogni i prendo i nodi raggiungibili da quella i è come prendere ogni arco.

Prossima operazione: la seconda $\sum_{k \in K} \sum_{j, i \in A} \lambda_i^k x_{ji}^k$ viene rinominata cambiando $j \rightarrow i$ e $i \rightarrow j$ così diventa:

$$\sum_{k \in K} \sum_{i, j \in A} \lambda_j^k x_{ij}^k \quad (2.18)$$

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} \lambda_i^k x_{ij}^k - \sum_{k \in K} \sum_{i,j \in A} \lambda_j^k x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k * b_i^k \quad (2.19)$$

Arrivati a sto punto si raccoglie:

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + \sum_{k \in K} \sum_{i,j \in A} (\lambda_i^k - \lambda_j^k) x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.20)$$

↓

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.21)$$

Dato che l'ultima parte non ha variabili decisionali può essere tirata fuori in quanto nella ricerca del min non cambia niente.

$$L(\lambda) = \min \sum_{k \in K} \sum_{i,j \in A} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.22)$$

Ci troviamo ora davanti a un problema che si basa unicamente su i valori di un arco e i nodi confinanti quindi è come avere numero archi problemi.

$$LR_{i,j \in A}(\lambda) = \min \sum_{k \in K} c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.23)$$

$$L(\lambda) = \sum_{i,j \in A} LR_{i,j}(\lambda) - \sum_{k \in K} \sum_{i \in N} \lambda_i^k b_i^k \quad (2.24)$$

$LR_{i,j}(\lambda)$ indica la soluzione ottima per quel sotto problema i, j Ma come si risolve $LR_{i,j}(\lambda)$?

Creo una variabile d_{ij}^k che mi indica il risultato di $LR_{i,j}(\lambda)$ per il flusso k cioè:

$$\min c_{ij} e_{ij}^k + (\lambda_i^k - \lambda_j^k) x_{ij}^k \quad (2.25)$$

Ho 2 scelte: se non attivo l'arco allora mi costa 0 perché $e_{ij}^k = 0$ e $x_{ij}^k = 0$ per il vincolo (27) Se invece lo attivo d_{ij}^k mi indica il suo costo e se è < 0 allora mi conviene attivarlo, se invece maggiore di 0 non mi conviene attivarlo. Per il calcolo di d_{ij}^k c_{ij} lo abbiamo già: $e_{ij}^k = 1$ perché vogliamo vedere il costo se è attivo altrimenti sarebbe $d_{ij}^k = 0$. x_{ij}^k dipende da $(\lambda_i^k - \lambda_j^k)$ perché se $(\lambda_i^k - \lambda_j^k) > 0$ noi vogliamo minimizzare e quindi ridurre al minimo la somma positiva quindi x_{ij}^k dovrà assumere il minore valore possibile che per lui è 1 altrimenti se $(\lambda_i^k - \lambda_j^k) < 0$ allora vogliamo massimizzare questo valore negativo perché ci aiuterà a minimizzare il problema globale e quindi vogliamo che x sia il massimo valore da lui possibile che è $NodiTarget_k$ e questo si deduce dal vincolo 27. Ora che sappiamo in un arco per quali flussi conviene attivare con i costi penalizzati ci

troviamo davanti a un'altro problema cioè quello della capacità dell'arco, può darsi che l'arco non possa far passare tutte le informazioni. Per fortuna si può notare come esso sia un problema di Knapsack quindi abbiamo uno zaino(l'arco) con una capacità massima u e degli oggetti che vogliamo inserire k con il loro peso a e il loro valore d . Nel nostro caso le d saranno da $\ast -1$ perché altrimenti non ha senso che il loro vantaggio sia negativo. Il knapsack lavora con un vantaggio positivo.

2.5.1 Aggiornamento Lambda

Per aggiornare le λ usiamo il metodo del subgradiente. Prendo il vincolo che avevo rilassato cioè:

$$\sum_{j \in L_i} x_{ij}^k - \sum_{j \in R_i} x_{ji}^k - b_i^k = 0 \quad (2.26)$$

definisco il subgradiente come:

$$s_i^k = \sum_{j \in L_i} x_{ij}^k - \sum_{j \in R_i} x_{ji}^k - b_i^k \quad (2.27)$$

Questo mi indica di quanto stiamo sforando il vincolo che dovrebbe tendere a 0. Definiamo s_i^k dipendente da k e i perché il vincolo 25 è dipendente da quelle 2 cose mentre le j le somma tutte e ha senso che sia per un nodo i in quanto anche λ è legata a un nodo non a un arco. La soluzione ottima è quella che rispetta tutti i vincoli anche quello rilassato, se non trovo una soluzione che riesce a essere ottima allora in vincolo rilassato non viene rispettato. Una volta che abbiamo definito s_i^k le calcoliamo tutte insieme ai x_{ij}^k e andiamo ad aggiornare λ_i^k facendo $\lambda_i^k = \lambda_i^k + \theta s_i^k$. Per scegliere il passo θ utilizziamo:

$$\theta = \alpha \frac{\text{UpperBound} - \text{LowerBound}}{\|s^2\|} \quad (2.28)$$

Scegliamo proprio questi parametri per garantire all'inizio salti grandi per convergere in fretta e poi salti più piccoli per evitare la divergenza: α è un moltiplicatore generale che viene dimezzato ogni volta che il nostro LowerBound non aumenta per tot iterazioni. UpperBound – LowerBound ha una logica teorica dietro: io so che il mio ottimo sta tra l'upper bound e il lower bound quindi se loro si avvicinano molto devo fare un lavoro di precisione per trovare l'ottimo invece se sono distanti allora posso permettermi salti più grandi. $\|s^2\|$ è la norma di tutti i s_i con k fissato. Se gli s sommati hanno un valore alto vuol dire che stiamo sforando di tanto e quindi dividendo per $\|s^2\|$ riduciamo il passo evitando λ esagerate o che il passo diventi spropositato. Stessa cosa al contrario, se i subgradienti sono piccoli allora la norma può diventare molto piccola e dividendo per essa possiamo aumentare un po' il passo che altrimenti per gli altri 2 valori rischierebbe di muoversi troppo lentamente quando siamo vicini all'ottimo.

3 Implementazione

3.1 Upper bound

3.2 Knapsack

3.3 Scelta step

3.4 Possibile algoritmo di mostraggio risultato o distribuito o correzione di quello del tipo